Programmation Orientée Objet (POO) à destination de la robotique

Travaux Pratiques : Cartographie et Grille d'Occupation

Enseignant : Adam GOUGUET
Mail : adam.gouguet@imt-nord-europe.fr
Année : 2025–2026

Table des matières

1 Objectifs du TP **1**

2 La classe `GrilleOccupation` **2**
2.1 Le constructeur et l'encapsulation 2
2.2 Conversions et Accesseurs (Getters / Setters) 2

3 La classe `Cartographe` **2**
3.1 Composition de la classe 2
3.2 Mise à jour via le Lidar 3

4 Affichage et Validation **4**

1 Objectifs du TP

En robotique mobile, un robot a besoin de comprendre son environnement. L'objectif de ce TP est de modéliser une carte 2D.

Afin de respecter les bonnes pratiques de la Programmation Orientée Objet (POO), nous allons séparer notre programme en deux classes distinctes :

— `GrilleOccupation` : Une structure de données qui gère uniquement le stockage spatial, la matrice en mémoire et les conversions de coordonnées.

— `Cartographe` : L'algorithme qui utilise les données du capteur Lidar pour modifier la grille via l'algorithme de lancer de rayon (*Raycasting*).

Ce concept, où une classe complexe (`Cartographe`) utilise les services d'une autre classe (`GrilleOccupation`), s'appelle la **Composition**.

2 La classe GrilleOccupation

Créez un fichier `grille_occupation.py`. Cette classe représente la carte physique. Chaque cellule de la grille possèdera un état : **-1** (Inconnu), **0** (Libre), ou **1** (Occupé).

2.1 Le constructeur et l'encapsulation

1. Écrivez le constructeur `__init__(self, largeur_m, hauteur_m, resolution=0.1)`. Il doit calculer la taille de la matrice (`nx`, `ny`) et définir le centre du repère (`origin_x`, `origin_y`).
2. Initialisez la grille sous forme d'un tableau numpy. **Attention à l'encapsulation !**

2.2 Conversions et Accesseurs (Getters / Setters)

Puisque `_grid` est privée, nous devons fournir des méthodes pour interagir avec elle de manière sécurisée.

Travail à faire : Implémentez les trois méthodes suivantes :

```
    def coord2index(self, x, y):
        """ Convertit (x,y) en metres vers (ix, iy) en indices.
            Doit brider les valeurs (clip) pour ne pas sortir de la matrice ! """
        pass

    def get_cellule(self, ix, iy):
        """ Retourne l'etat de la cellule aux indices donnes. """
        return self._grid[ix, iy]

    def set_cellule(self, ix, iy, etat):
        """ Modifie l'etat de la cellule aux indices donnes. """
        self._grid[ix, iy] = etat
```

3 La classe Cartographe

Créez un second fichier `cartographe.py`. C'est ici que réside "l'intelligence" de la cartographie. Le cartographe a besoin d'une grille pour travailler.

3.1 Composition de la classe

Dans son constructeur, le `Cartographe` va prendre en paramètre une instance de `GrilleOccupation` et la sauvegarder en tant qu'attribut.

```
class Cartographe:
    def __init__(self, grille: GrilleOccupation):
        self.grille = grille # Relation de composition
```

3.2 Mise à jour via le Lidar

Lorsqu'on demande au cartographe de se mettre à jour, il récupère les rayons du Lidar et applique l'algorithme de Bresenham pour dessiner les zones libres et les obstacles dans la grille.

```
def mise_a_jour(self, env, lidar):
    rays = lidar.get_rays_world(env)
    for x1, y1, x2, y2, dist in rays:
        self._bresenham_update(x1, y1, x2, y2, dist, lidar.max_range)
```

Travail à faire : Implémentez la méthode privée `_bresenham_update`. **Rappel :** Vous n'avez plus accès à `self._grid` directement ici ! Vous devez utiliser `self.grille.coord2index()`, `self.grille.get_cellule()` et `self.grille.set_cellule()` pour interagir avec la carte.

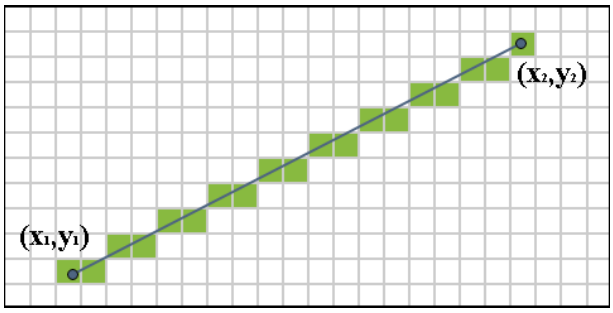

FIGURE 1 – Algorithme de tracé de segment de Bresenham

Algorithm 1 Algorithme de Bresenham

Require: Points (x_0, y_0) et (x_1, y_1)

$dx \leftarrow |x_1 - x_0|$
$dy \leftarrow |y_1 - y_0|$
$sx \leftarrow$ **if** $x_0 < x_1$ **then** 1 **else** -1
$sy \leftarrow$ **if** $y_0 < y_1$ **then** 1 **else** -1
$err \leftarrow dx - dy$
while true **do**
  tracer le point (x_0, y_0)
  if $x_0 = x_1$ **and** $y_0 = y_1$ **then**
    break
  end if
  $e2 \leftarrow 2 \times err$
  if $e2 > -dy$ **then**
    $err \leftarrow err - dy$
    $x_0 \leftarrow x_0 + sx$
  end if
  if $e2 < dx$ **then**
    $err \leftarrow err + dx$
    $y_0 \leftarrow y_0 + sy$
  end if
end while

4 Affichage et Validation

L'affichage est une fonctionnalité propre à la donnée spatiale. Ajoutez une méthode `dessiner_grille` à votre vue prenant en argument une variable de type `GrilleOccupation`.

Test final (Script principal) : Dans votre fichier principal (`main.py`), testez l'architecture complète :

```
# 1. Creation de la memoire spatiale
ma_grille = GrilleOccupation(largeur_m=10, hauteur_m=10, resolution=0.5)

# 2. Creation de l'algorithme associe a la grille
carte = Cartographe(ma_grille)

# 3. Boucle de simulation (fictive)
while true:
    ...
    carte.mise_a_jour(environnement, lidar)

    # 4. Affichage du resultat
    vue.dessiner(grille=ma_grille)
```

Résultat souhaité :

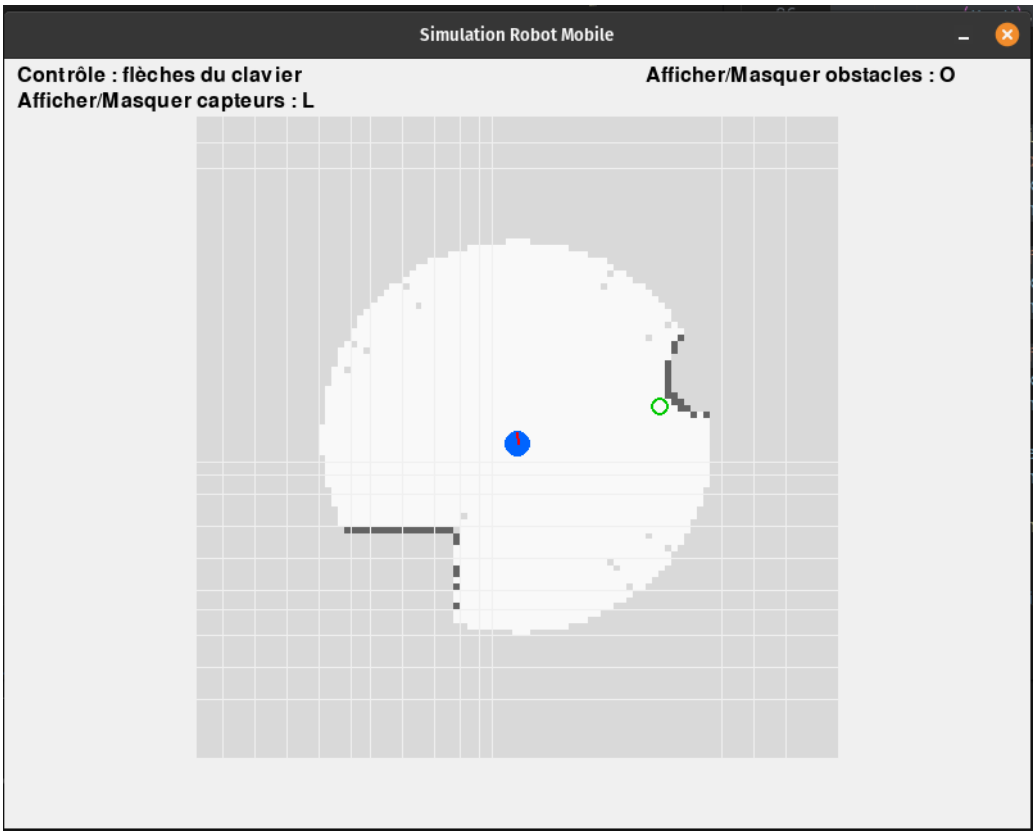

FIGURE 2 – Exemple de la cartographie dans la simultion python.